Catálogo PHP 8 orientado a objetos con formularios seguros y persistencia JSON

Rodrigo Ledezma Sanchez

Grupo: HTML

Universidad Privada Domingo Savio

Ingeniería de Sistemas

Tecnología Web II (SIS-0151)

Docente: M.Sc. Ing. Jimmy Nataniel Requena Llorentty

Actualización: 14 de octubre de 2026

Resumen

La entrega reúne cinco laboratorios y una aplicación integrada de catálogo desarrollada con PHP 8. La solución registra, busca, edita y elimina productos, admite fotografías y conserva la información en un archivo JSON. El dominio se modeló mediante las clases Producto, Categoria y Catalogo, con tipos estrictos, promoción de propiedades y responsabilidades separadas. La seguridad incluye validación en el servidor, escape de salida, patrón POST/Redirect/GET, verificación del tipo real de las imágenes y almacenamiento fuera de la carpeta pública. Las trece pruebas funcionales y las diez pruebas específicas de orientación a objetos finalizaron correctamente.

Palabras clave: PHP 8, programación orientada a objetos, formularios, JSON, seguridad web.

Introducción

El objetivo de la actividad fue construir una aplicación pequeña pero completa sin utilizar todavía una base de datos. Los laboratorios 1 a 4 aportaron progresivamente el entorno de ejecución, las funciones del inventario, la recepción y validación de formularios y la gestión segura de archivos. La versión final reorganiza esas capacidades alrededor de objetos del dominio y mantiene una separación clara entre modelo, persistencia, controladores y presentación. El laboratorio 5 se conserva además como proyecto independiente con su modelo encapsulado, demo y verificador oficial de 14 pruebas.

Diseño orientado a objetos

Clase Producto

Producto representa una entidad del catálogo. Su constructor emplea promoción de propiedades con tipos explícitos y propiedades de solo lectura. También transforma sus datos a un arreglo serializable y usa match para clasificar el nivel de stock. La clase no

genera HTML ni conoce la ubicación del JSON.

Clase Categoria

Categoria concentra las opciones válidas del dominio: Miel, Derivados y Cera. Al impedir la creación de categorías arbitrarias, evita que una regla importante quede repetida y dispersa en las vistas.

Clase Catalogo

Catalogo encapsula la ruta de persistencia y convierte las filas JSON en objetos Producto. Sus métodos permiten recuperar la colección, buscar por código y guardar de forma centralizada. La escritura utiliza LOCK_EX, JSON_UNESCAPED_UNICODE y JSON_PRESERVE_ZERO_FRACTION.

Formularios y controles de seguridad

Las búsquedas utilizan GET porque no modifican información y pueden conservarse en la URL. El registro, la edición y la eliminación utilizan POST. Todo dato se valida otra vez en el servidor, incluso cuando el formulario contiene controles HTML. Los mensajes de error se asocian a sus campos y los valores se escapan mediante htmlspecialchars antes de incorporarse a la salida.

Un registro válido responde con 303 See Other y aplica el patrón POST/Redirect/GET. De este modo, al actualizar la página no se reenvía el formulario. Las entradas inválidas responden con 422; las fotos o productos inexistentes con 404; y un intento de eliminación por GET con 405 y la cabecera Allow: POST.

Subida segura de fotografías

La aplicación no confía en el nombre, la extensión ni el tipo MIME declarado por el navegador. finfo lee el contenido real, getimagesize confirma que sea una imagen y una lista permitida limita los formatos a JPEG, PNG y WebP. Cada archivo recibe un nombre criptográficamente aleatorio, se limita a 2 MB y se guarda fuera de public. foto.php valida el formato del nombre antes de entregar el contenido con la cabecera X-Content-Type-Options: nosniff.

Resultados de verificación

Conjunto	Resultado	Aspectos comprobados
Laboratorio 1	12 de 12	PHP dinámico, escape, formato y estructura
Laboratorio 2	12 de 12	Funciones tipadas, inventario, cadenas y reporte
Laboratorio 3	13 de 13	Formulario, validación, respuestas HTTP, PRG y JSON
Laboratorio 4	13 de 13	Fotos seguras, edición, eliminación y rutas
Laboratorio 5	14 de 14	Validación en constructor, stock privado, excepciones, duplicados e inventario

Conjunto	Resultado	Aspectos comprobados
Integración POO	10 de 10	Clases, promoción, encapsulamiento, match, nullsafe y persistencia

Además de ejecutar los verificadores, se realizó análisis sintáctico de todos los archivos PHP. Las pruebas restauran el estado inicial para ser repetibles y verifican efectos reales en el JSON y en la carpeta privada de fotografías.

Repositorio del proyecto

El código fuente, las hojas de trabajo, las bitácoras y las evidencias de los cuatro laboratorios están publicados en el siguiente repositorio de GitHub:

https://github.com/ROXIAN120/Actividad-1-tecnologia-web-2

La rama principal contiene el índice de entrega y enlaces directos a las ramas laboratorio-1, laboratorio-2, laboratorio-3 y laboratorio-4. Cada rama conserva su implementación y sus capturas. La actualización local agrega laboratorios/laboratorio-1 a laboratorio-5 y las tres evidencias del laboratorio 5. Esa ampliación se entrega también en el paquete local; su publicación en GitHub está pendiente.

Conclusiones

La aplicación cumple los tres ejes de la rúbrica: utiliza estructuras modernas de PHP con tipos estrictos, implementa formularios y archivos con controles verificables, y modela el dominio con clases cohesionadas. La separación lograda permite sustituir posteriormente el archivo JSON por una base de datos sin trasladar la lógica de persistencia a las vistas. Como mejora futura se recomienda incorporar protección CSRF, autenticación y transacciones en una base de datos relacional.

Evidencias de los cinco laboratorios

Laboratorio 1 — Catálogo dinámico con PHP

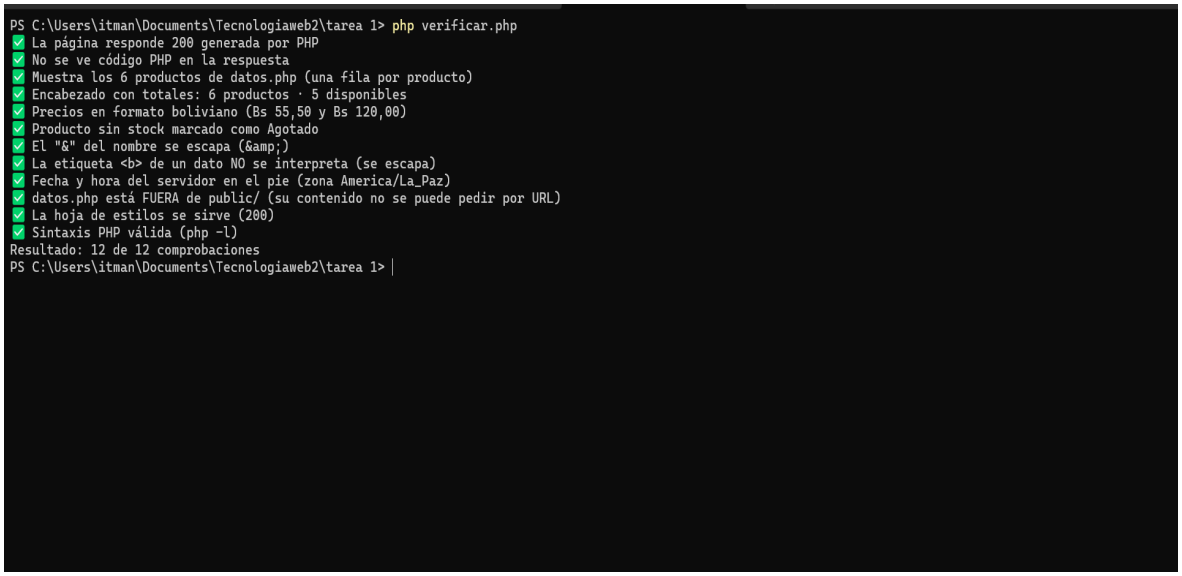


Figura 1. Verificador del Laboratorio 1: 12 de 12 comprobaciones.

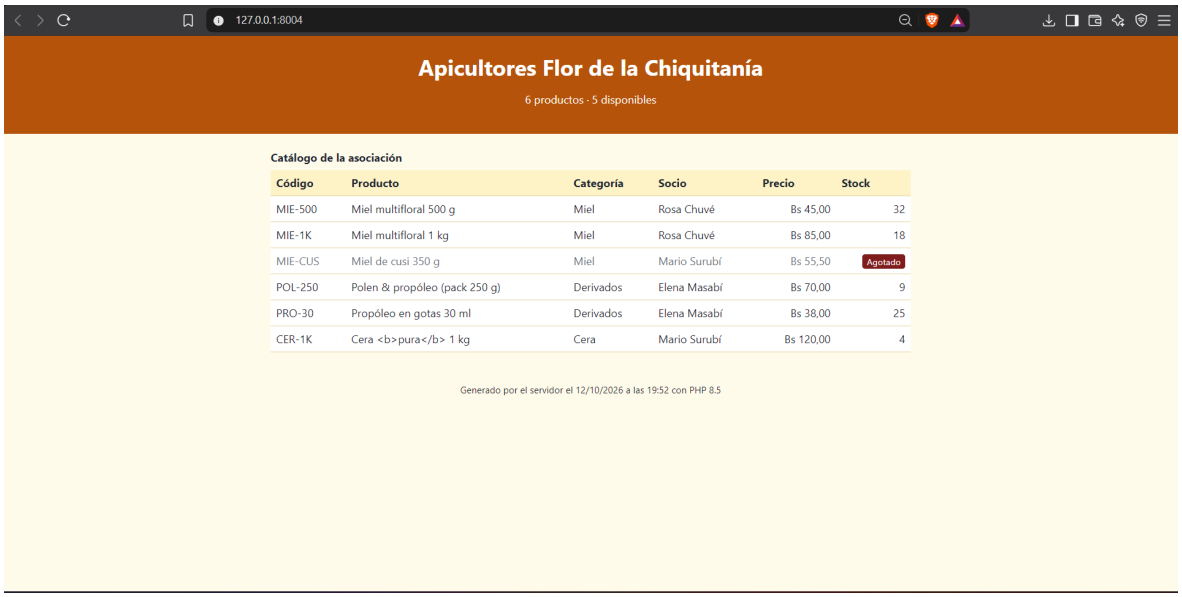


Figura 2. Catálogo generado dinámicamente y salida escapada.

Laboratorio 2 — Funciones tipadas para el inventario

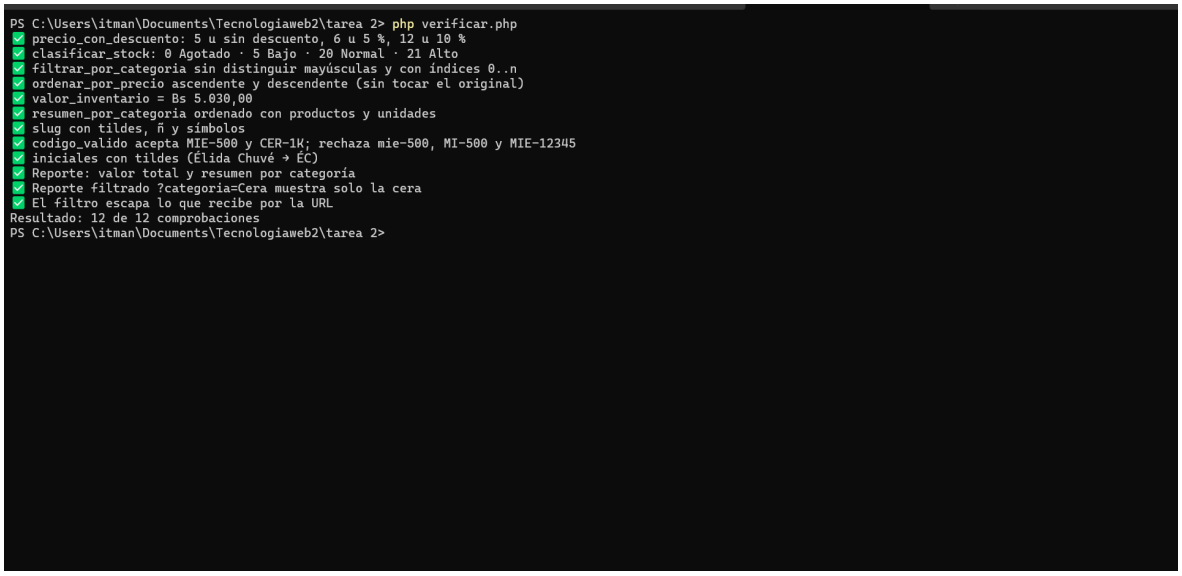


Figura 3. Verificador del Laboratorio 2: 12 de 12 comprobaciones.



Figura 4. Reporte de inventario con resumen y cálculos.

Laboratorio 3 — Formulario, validación, PRG y JSON

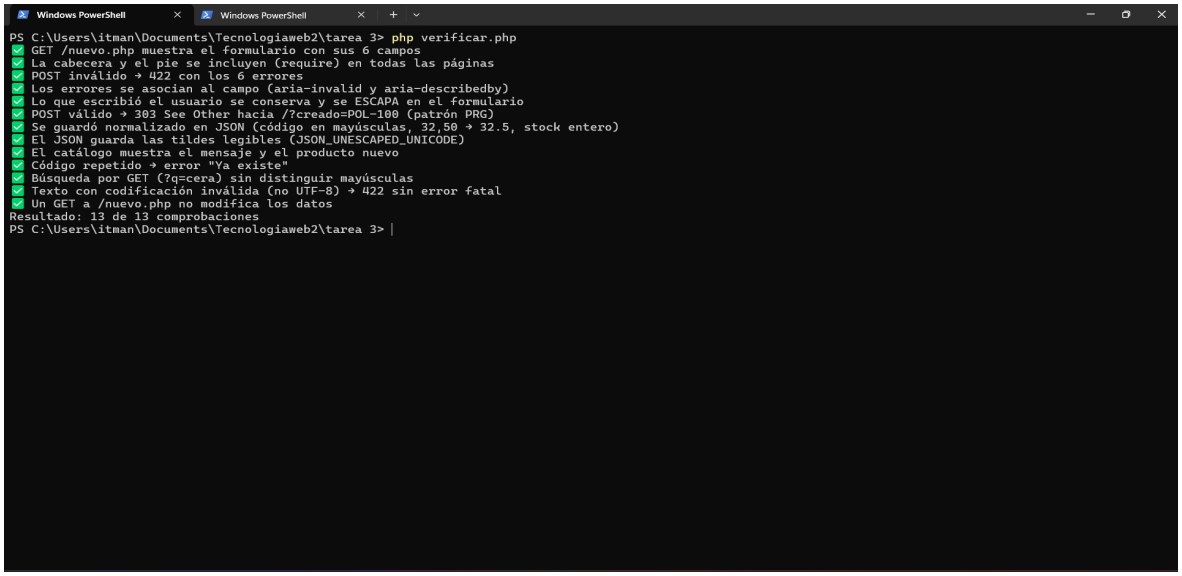


Figura 5. Verificador del Laboratorio 3: 13 de 13 comprobaciones.

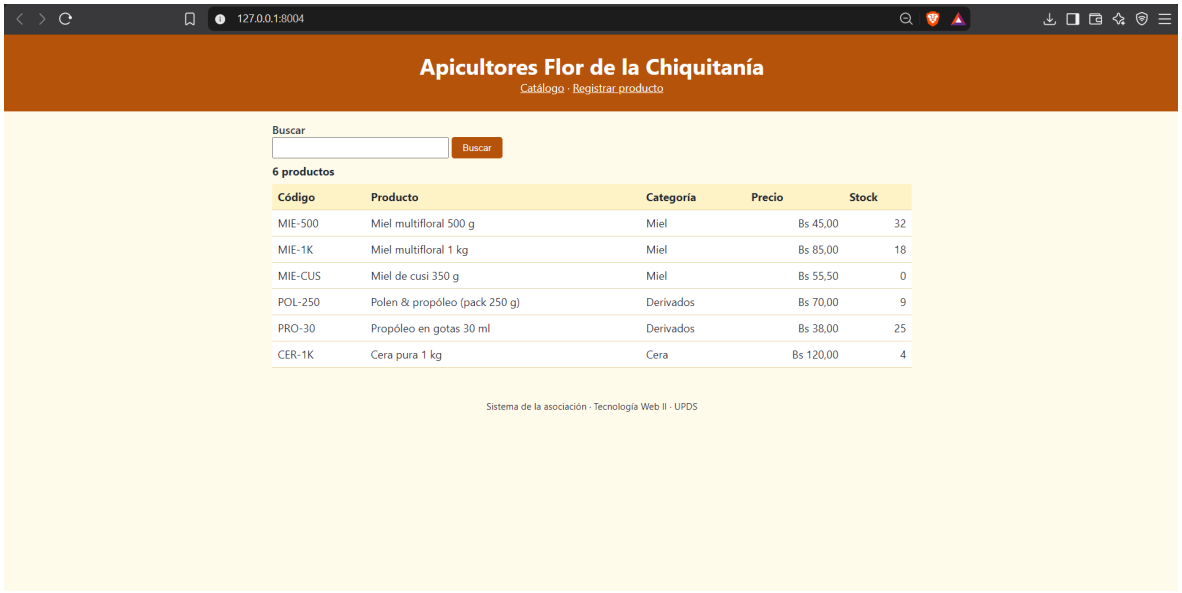


Figura 6. Catálogo con búsqueda y formulario de registro.

Laboratorio 4 — Fotografías seguras, edición y eliminación

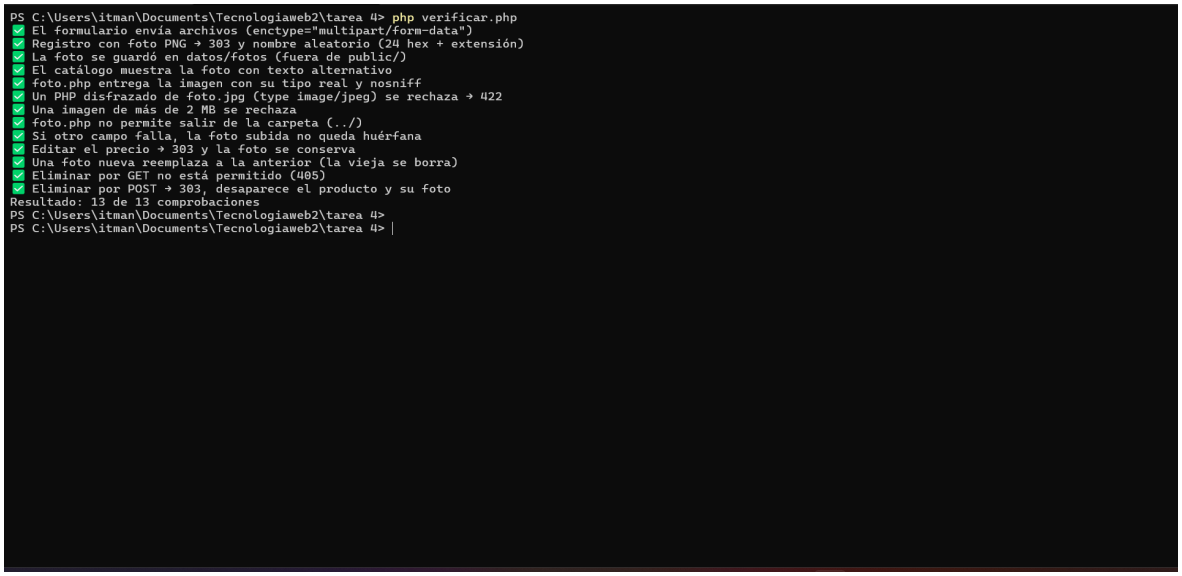


Figura 7. Verificador del Laboratorio 4: 13 de 13 comprobaciones.

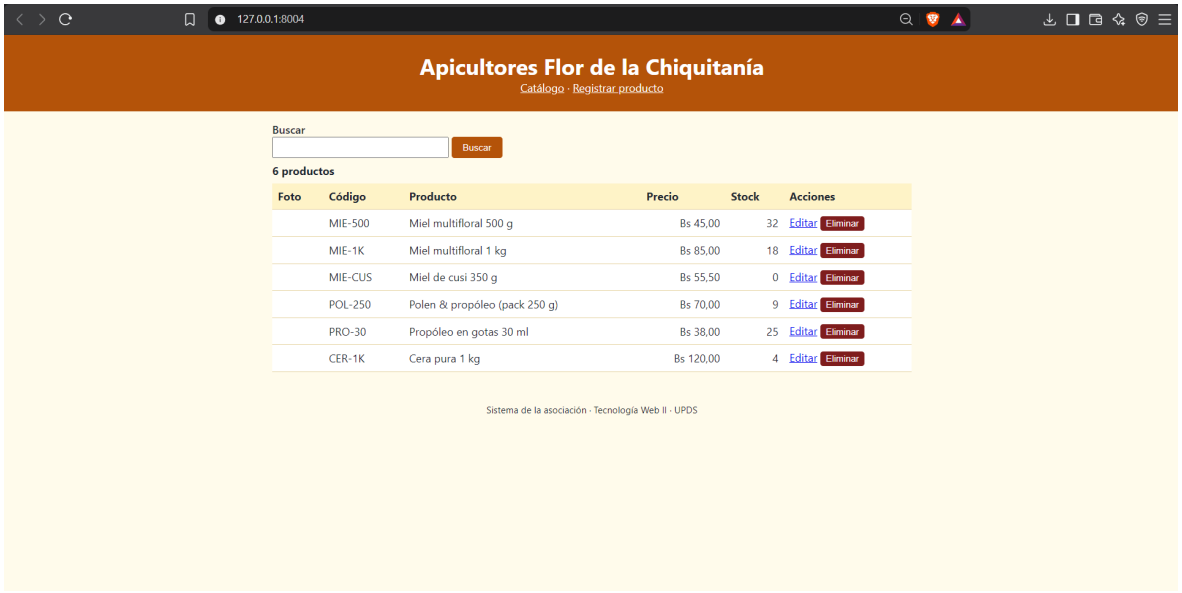


Figura 8. Catálogo con acciones seguras de edición y eliminación.

Laboratorio 5 — Modelo que protege sus reglas

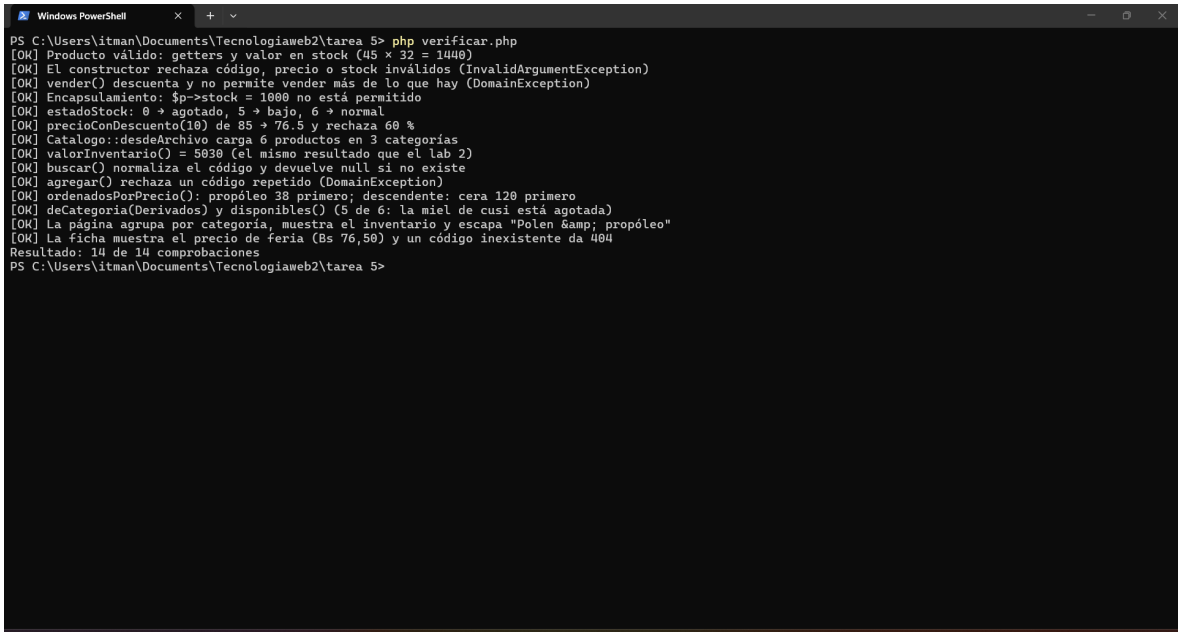


Figura 9. Verificador oficial del Laboratorio 5: 14 de 14.

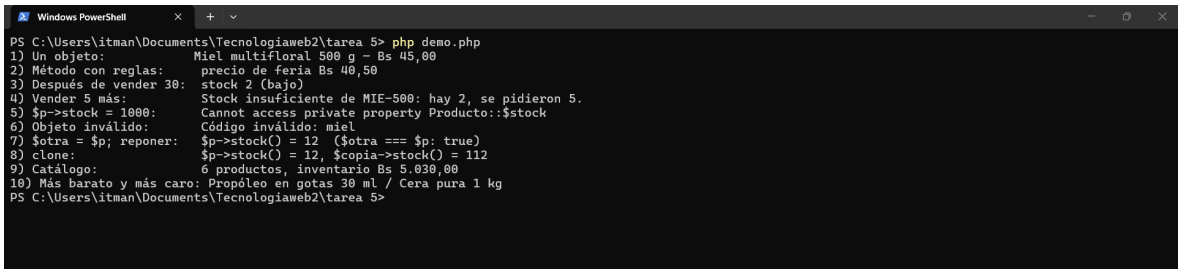


Figura 10. Demo: excepciones previstas, referencias y clonación.

La demo conserva 12 unidades en el original y 112 en la copia. El inventario es Bs 5.030,00. Los mensajes de los pasos 4 a 6 demuestran que se bloquean operaciones inválidas.

Laboratorio 5 — Catálogo con objetos



Figura 11. Seis productos, cinco disponibles y agrupación por categoría.

Este proyecto conserva el inicio del docente: no incluye el formulario de ventas persistidas ni la clase Socio de los retos opcionales. Producto valida su constructor, mantiene stock privado y solo lo modifica mediante vender() y reponer(). Catalogo rechaza códigos duplicados, ordena por precio y suma el valor en stock. Es una implementación independiente de la aplicación integrada descrita anteriormente; ambas se adjuntan para no confundir sus APIs.

Integración orientada a objetos

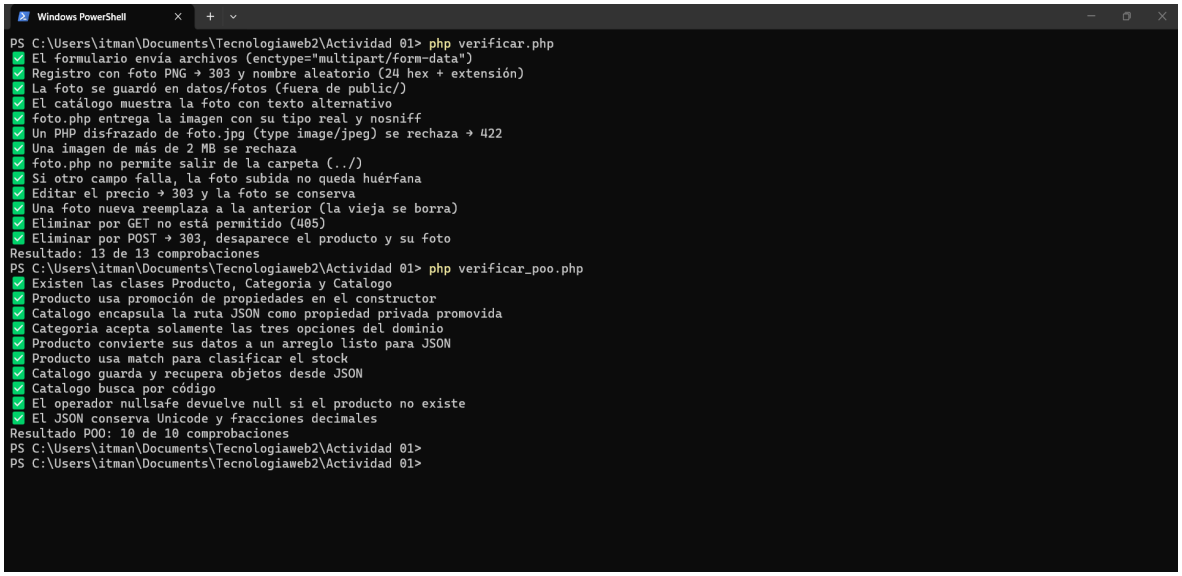


Figura 12. Pruebas funcionales 13/13 y comprobaciones adicionales POO 10/10.

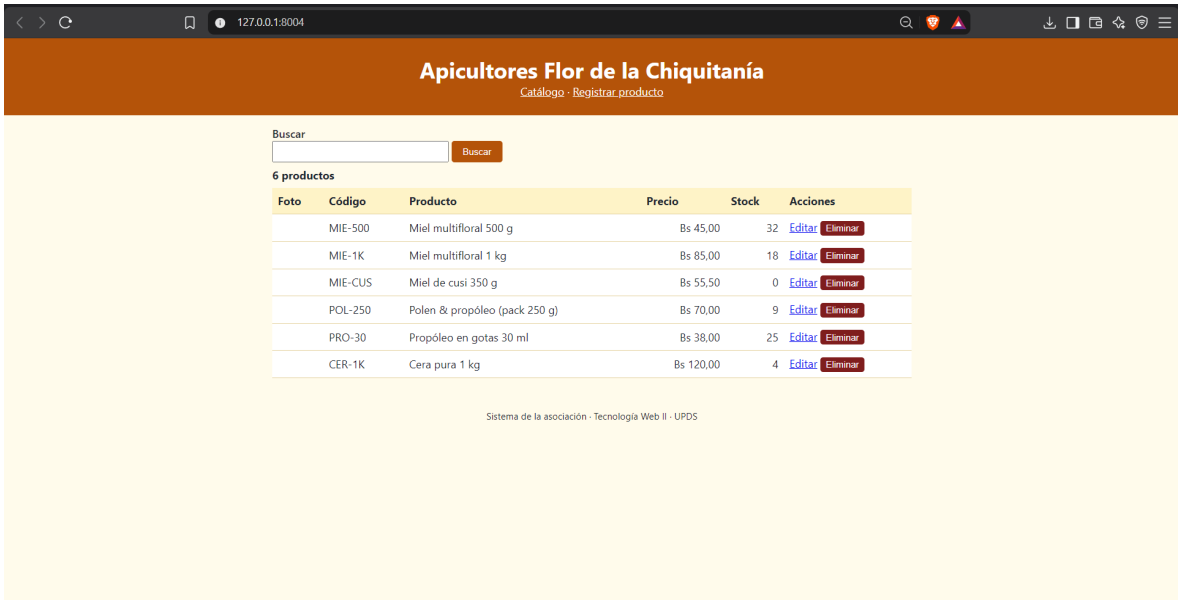


Figura 13. Aplicación final integrada ejecutándose en PHP 8.

Referencias

PHP Documentation Group. (2026). PHP manual. https://www.php.net/manual/es/

OWASP Foundation. (2025). File upload cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

Universidad Privada Domingo Savio. (2026). Guías de laboratorio 1–5 de Tecnología Web II [Material de clase].